

Riyad al-Salihin v2 app

A guide to every folder and file, how the app works, and a review of its size, design and UX

Branch	`v2-app` (React Native with Expo), commit `bd1546f`, 2026-11-04 (“v2 app P5: icon, splash, large text, screen readers”)
Repository	`nadernour/riyad`
Prepared	2026-11-06
Stack	Expo SDK 58, React Native 0.88.1, React 19.3, TypeScript 6, expo-router (drawer), expo-sqlite (FTS5), Reanimated 4.7, Gesture Handler 3.3

Contents

1. Summary of findings
2. Folder structure
3. Every file, folder by folder
4. How the app works
5. Review: why the Android APK is about 100 MB, and how to shrink it
6. Review: design and colours
7. Review: user experience
8. Review: code, build and release
9. Suggested order of work
10. Appendix: how the numbers were measured

1 Summary of findings

The `v2-app` branch is a rewrite of the Kotlin v1 app as one Expo (React Native) app for Android and iPhone. It reads all 1,898 hadith from a bundled SQLite database and works fully offline. The code is in good shape: small, well commented, and tested. On the checked-out commit the 65 unit tests pass, and `npm run typecheck` and `npm run lint` report no problems. The README says its 7 Maestro device flows pass on the iPhone simulator and a Galaxy A17.

The review found these main points. Each one is covered in detail later.

1. **The 100 MB APK is mostly native code built four times, not your app.** The JavaScript bundle, the database and the fonts add up to only about 8 MB of the APK. The rest is React Native's native libraries. Expo builds them for 4 CPU types by default (`armeabi-v7a, arm64-v8a, x86, x86_64`), and a universal APK carries all four, stored uncompressed. A phone uses only one of them. The Play Store build (an app bundle, `.aab`) already sends each phone only its own CPU type, so a user's download should be roughly 15–25 MB. For test APKs, build only `arm64-v8a`, which should bring them to about 30 MB. Section 5 has the exact settings.
2. **The colours don't come from the logo.** The icon is gold calligraphy (`#FFD700`) on near-black (`#111318`). The app's accent is teal green (`#0B6E5F` light, `#5CC9B4` dark), the splash screen shows the logo in green, and the dark background is a green-black (`#0F1615`). No screen uses the gold. (v1 didn't either: it used Material Design's default blue.)
3. **One colour does every job.** The token `colors.prophet` colours the Prophet's words ﷺ, and it is also used for section headings, the active menu item, list numbers, icons, buttons, spinners and the About title: 22 places in 10 files. So in the reader the green no longer means only one thing.
4. **Proposed palette: "Gold & ink".** It uses the logo's near-black as the dark background, a warm paper white for light mode, and gold as the accent. On a light background the gold has to be a darker bronze-gold: the logo's `#FFD700` on white has a contrast of only 1.36:1, which is unreadable. There is a separate token for the Prophet's words. All text colours pass WCAG AA contrast. The mockups are in Section 6.
5. **UX gaps worth fixing before release:** search results don't show the matching text. Tapping anywhere toggles full screen, which will clash with the planned "tap a word" and "tap a narrator" features. The hadith text can't be selected or copied. The reader shows no progress through the book and has no quick way to jump to a hadith number. Removing a bookmark asks for confirmation every time, where an undo would be quicker.
6. **Release details to settle before P6:** the generated Android manifest asks for permissions an offline app doesn't need: storage, "display over other apps", vibrate and internet. `versionCode` is set by hand. The About screen links to v1's GitHub project and email. Error messages show to users in English. The iPad layout isn't planned. `assets/favicon.png` is Expo's blue placeholder.

2 Folder structure

The repository has three parts: **the Expo app** at the root (`src/`, `assets/`, `database/` and the config files), **the content pipeline** in `book_data/`, which produces `database/riyad.db`, and **the old v1 app** in `v1/`. Metro, ESLint, TypeScript and Jest are all set to ignore `book_data/` and `v1/`.

```
riyad/  (branch v2-app)
├── src/                  the app's TypeScript code
│   ├── app/              screens; Expo Router turns every file here into a route
│   │   └── (drawer)/     the six screens inside the side-menu (drawer) navigator
│   ├── reader/           the hadith reader: pager, page, styled text, header, sharing
│   ├── db/               data layer: content queries, user database, bookmarks, settings
│   ├── components/       shared views: Text, Icon, HadithRow, SideMenu
│   ├── search/           Arabic text simplification for search
│   ├── text/             Arabic number agreement for counts of hadith and chapters
│   └── theme/            colours, fonts, text-scaling limits
├── assets/               app icon, adaptive-icon layers, splash images
│   └── fonts/            Scheherazade New (hadith text) and Cairo (interface)
├── database/             riyad.db, the SQLite database the app ships with, and its docs
├── plugins/              a custom Expo config plugin (Android deep-link fix)
├── scripts/              Node scripts: the test-build "preview" database with v1 commentary
├── .maestro/             end-to-end device tests (Maestro flows)
├── book_data/            the content pipeline (owner's Debian machine only)
│   ├── content/          the source of truth: the checked text, one JSON per book
│   │   ├── hadith/       20 books → chapters → hadith (text as speaker-tagged pieces)
│   │   ├── matn/         step-4 decisions: corrections made against the printed book
│   │   └── audit/        30 audited hadith (the test set)
│   ├── docs/             method and rules of each content step
│   └── tools/            Python tools: OCR, comparison, numbering, building riyad.db
└── v1/                   the old Kotlin / Jetpack Compose app, kept for reference
    ├── app/              v1's Android source, resources and its 15 MB database
    ├── docs/screenshots/ v1 screenshots
    ├── dp_update/        the script that turned the original HTML database into v1's
    └── gradle/ ...       v1's Gradle build
```

Folder	Purpose	Used by
`src/app/`	Screens. With Expo Router every file is a route; `_layout.tsx` files define navigators. The root layout loads fonts and the database; `(drawer)/` is a route group (the brackets keep it out of the URL) holding the 6 screens of the side menu.	App
`src/reader/`	Everything about reading one hadith per page: the custom right-to-left pager, the page layout, the speaker-coloured text, the overlay header, “last read” memory, sharing.	App
`src/db/`	All data access. `riyad.db` (read-only content) through async queries; `user.db` (bookmarks, settings) through small synchronous queries; React stores so every screen sees the same bookmarks and settings.	App
`src/components/`	Views used by several screens.	App
`src/search/`, `src/text/`	Pure functions: Arabic normalisation for search, and Arabic number agreement.	App
`src/theme/`	The colour tokens for light and dark, the font names, line height, and how far the phone’s text size may enlarge text.	App
`assets/`	Images and fonts bundled into the app.	App
`database/`	The built content database, its documentation and the shared search test cases. Built by `book_data/tools/build_db.py`, never edited by hand.	App + pipeline

Folder	Purpose	Used by
`plugins/`	Expo config plugins that patch the generated native projects during `expo prebuild`.	Build
`scripts/`	Node scripts run by hand (`npm run preview-db`).	Build / test builds
`.maestro/`	Device tests that tap and swipe through a running test build.	Testing
`book_data/`	The content: the text checked against the printed book, the tools that check it, and the decisions. Runs only on the owner's Debian machine.	Content pipeline
`v1/`	The released v1 Android app (Kotlin, Jetpack Compose). The reference for v1 behaviour and an input to the content pipeline. To be deleted after v2 ships.	Reference

Not in git, created when you work on it: `node_modules/`, `ios/` and `android/` (generated by `npx expo prebuild` from `app.json`; native settings go in `app.json`, never in these folders), `.preview/` (the test-only database with v1 commentary), and `book_data/.cache/` and `book_data/.venv/` (book scans, OCR, Python environment).

3 Every file, folder by folder

All 259 files tracked on `v2-app`, grouped by folder. Sizes are rounded. Where a folder holds many files of the same kind (books, audit records, v1 icon densities), they are described together, with each file named.

Root: configuration and documentation

File	Size	What it is for
`README.md`	17 KB	The project's plan and status: what the app shows (the hadith, word meanings and benefits from *Nuzhat al-Muttaqin*), the feature list, how the content is checked, the app's architecture, how to run, test, build and release it, the phase checklist (P0–P6) and the repository layout.
`CLAUDE.md`	4 KB	Instructions for Claude Code sessions working in the repo: the two people and their machines, SDK 58 documentation rules, `npx expo install`, run lint/typecheck/test before calling work done, and the owner's recorded decisions.
`LICENSE`	1 KB	MIT licence. It covers the code only, not the text of the books.
`.gitignore`	1 KB	What git ignores: editor files, `local.properties` (signing passwords), the pipeline's caches, Expo outputs, the generated `ios/` and `android/`, signing files (except v1's key) and `.preview/`.
`package.json`	2 KB	npm manifest: the dependencies (Expo SDK 58 modules, React Native 0.88.1, Reanimated, Gesture Handler, Screens, Safe Area), the dev tools (Jest, TypeScript, ESLint), the scripts (`start, android, ios, test, lint, typecheck, submit, preview-db`), Jest settings and Node ≥ 22.13.
`package-lock.json`	725 KB	The exact version of every installed package, so installs are reproducible.
`app.json`	1.5 KB	The Expo app configuration: name «رياض الصالحين», version 2.0.0, portrait only, icon, iOS bundle id with tablet support, Android package `com.nad.riyadalsalheen` with `versionCode` 2, adaptive icon on `#111318`, and the plugins: router, status bar, SQLite, fonts, localization (forces right-to-left), splash screen (images and colours), assets, and the local link plugin.
`app.config.js`	0.8 KB	Wraps `app.json`. Every build that isn't `APP_VARIANT=production` becomes a *test build*, named «رياض الصالحين (تجربة)» with the id `….dev` and the link scheme `riyadalsalheen-dev`, so it installs next to the store app.
`eas.json`	0.6 KB	EAS Build and Submit profiles. `development`: a development client for internal testers. `preview`: an internal Android **APK**. `production`: sets `APP_VARIANT=production` and builds an Android App Bundle. `appVersionSource: local` means the version numbers come from `app.json`.
`metro.config.js`	2 KB	Metro (the JS bundler) settings. It bundles `.db` files as assets and keeps Metro out of `book_data/` and `v1/`. In test builds it swaps `database/riyad.db` for `.preview/riyad.db`, but only while that preview is up to date (checked by a SHA-256 stamp), and never in production.
`metro.config.test.js`	3 KB	Jest tests for the Metro config: the asset extension, the blocked folders, and the four preview-database cases (built, absent, out of date, production).
`eslint.config.js`	0.3 KB	ESLint flat config from `eslint-config-expo`, ignoring `dist, book_data, v1, android, ios`.

File	Size	What it is for
`tsconfig.json`	0.4 KB	TypeScript config: strict mode, the `@/…` → `src/…` import alias, Jest and Node types for tests.
`tsconfig.app.json`	0.4 KB	A second, app-only TypeScript check without Node types, so code that would only run in Node (`Buffer`, `fs`) fails before it reaches a phone. `npm run typecheck` runs both.

src/app: screens

File	Size	What it is for
`src/app/_layout.tsx`	2 KB	The root layout. It keeps the splash screen up, loads the five fonts, opens the content database (`ContentProvider`), applies the theme setting to the phone's own views (alerts, keyboard, status bar), and wraps everything in the gesture-handler root. The navigator mounts straight away and only its content waits, so a link that arrives during startup isn't lost.
`src/app/(drawer)/_layout.tsx`	3.5 KB	The side-menu navigator: the six screens with their Arabic titles and icons, the custom menu content, the header colours, the menu button placed correctly for right-to-left, and text-size limits on labels and titles.
`src/app/(drawer)/index.tsx`	5 KB	**The reader** (the home screen). It works out which hadith to show (from a link `/?id=N`, or the last one read), caps the number at the last hadith, saves the position, draws the `Pager` of `HadithPage`s under the `ReaderHeader`, toggles full screen (hiding the status and navigation bars), and turns pages for screen readers.
`src/app/(drawer)/books.tsx`	8 KB	**Books and chapters.** The 20 books expand to show their chapters. The book being read opens with its chapter marked and scrolled into view (with a frame-by-frame retry for an Android scrolling bug). Tapping a chapter opens its first hadith.
`src/app/(drawer)/search.tsx`	5 KB	**Search.** A text field that waits for a 250 ms pause in typing, focuses itself when the screen opens (after the menu finishes closing), shows a status line (hint, too short, no results, count, "first 100") and lists the results.
`src/app/(drawer)/bookmarks.tsx`	4 KB	**Bookmarks**, newest first. Remove one (with a confirmation), clear all (header button, with a confirmation), and an empty state with a hint.
`src/app/(drawer)/settings.tsx`	5 KB	**Settings.** Font size from 14 to 30 in steps of 2, with a live sample, and the theme (System, Light, Dark) as an accessible radio group.
`src/app/(drawer)/about.tsx`	6 KB	**About:** the book, the author, the app's features and text source, a notice in test builds, contact links (email, GitHub), the version and a closing du'a.

src/reader: the reader

File	Size	What it is for
`src/reader/Pager.tsx`	6.5 KB	A custom right-to-left pager built on Reanimated and Gesture Handler. All pages sit on one strip and only the strip's position is animated. Only the current page and two on each side are rendered. A drag past 25% or a fast flick turns the page, with resistance at the first and last page. It tells a tap apart from a drag and hides the off-screen pages from screen readers.
`src/reader/HadithPage.tsx`	6 KB	One hadith: chapter and number, short title, the text, then «لغة الحديث» (word meanings) and «أفاد الحديث» (benefits) when the database has them, the test-build label, and the printed page («نزهة المتقين، صفحة N»). It also adds the "next/previous hadith" actions for VoiceOver and TalkBack.

File	Size	What it is for
`src/reader/HadithText.tsx`	2 KB	The hadith text as one paragraph of nested spans, styled by speaker: the Prophet's words in `colors.prophet` and bold, other people's words in `colors.speech`. Justified, right-to-left, line height 2× the font size so the tashkeel isn't clipped.
`src/reader/ReaderHeader.tsx`	4 KB	The reader's bar: menu, the current book's title, search, share and bookmark. It sits over the pages and slides up for full screen.
`src/reader/lastHadith.ts`	0.7 KB	Reads and saves the last hadith read in `user.db`. If `user.db` fails it falls back to hadith 1.
`src/reader/openHadith.ts`	0.2 KB	Opens the reader at a hadith from any screen.
`src/reader/shareText.ts`	0.4 KB	The shared text: the title, the hadith, and «رياض الصالحين، الحديث رقم N».
`src/reader/shareText.test.ts`	0.7 KB	Test: the shared text holds the title, every piece of the text and the number.

src/db: data

File	Size	What it is for
`src/db/ContentProvider.tsx`	2.5 KB	Opens the bundled `riyad.db`. The copy on the phone is named after the file's hash (`riyad—<hash>.db`), so a new database in an app update is copied fresh and the old copy deleted. It checks the schema version and provides the `info` table, including the test-build "preview" flag.
`src/db/content.ts`	6 KB	Typed queries on `riyad.db`: schema check, info, count, one hadith with its pieces, meanings and benefits, books, chapters, hadith by a list of ids, number parsing (Western, Arabic-Indic and Persian digits), and full-text search with a limit of 100.
`src/db/content.test.ts`	4.5 KB	Runs the same SQL against the real `database/riyad.db` with Node's built-in SQLite: schema, numbering without gaps, books and chapters, ordering, and search (tashkeel, alif forms, partial words, digits, limits, quotes).
`src/db/hooks.ts`	3.3 KB	`useQuery`: runs a query when its key changes, drops late results, and can keep the previous result on screen. `useHadith`: a 30-hadith cache (least recently used) with shared loads in progress.
`src/db/bookmarks.ts`	2 KB	The bookmark store (`useSyncExternalStore`), so the reader's button, the bookmarks screen and the side menu change together. If `user.db` can't be read it retries after 5 s.
`src/db/settings.ts`	3 KB	The settings store: font size (14–30, step 2, default 24) and theme, with parsing and saving. `useTheme` and `useFontSize` re-render only when their own value changes.
`src/db/settings.test.ts`	1 KB	Tests for parsing and saving the settings.
`src/db/user.ts`	2.7 KB	The `user.db` schema as an append-only list of migrations, each run in a transaction, plus the SQL for settings and bookmarks.
`src/db/user.test.ts`	2 KB	Tests that migrating twice is harmless, that a failed migration rolls back, and the settings and bookmark ordering.
`src/db/userDb.ts`	0.6 KB	Opens `user.db` on first use and runs the migrations.
`src/db/testing.ts`	1 KB	Test-only adapters that make Node's `node:sqlite` look like expo-sqlite.

src/components, src/theme, src/search, src/text

File	Size	What it is for
`src/components/Text.tsx`	0.4 KB	Replaces React Native's `Text`, so the phone's text-size setting enlarges app text at most 1.5×.
`src/components/Icon.tsx`	2.5 KB	Icons named by meaning, mapped to SF Symbols on iPhone and Material Symbols on Android (expo-symbols). Hidden from screen readers. Corrects for Android's font scaling.
`src/components/HadithRow.tsx`	2 KB	A hadith in a list: number, title, and book and chapter. It can have an action button and a one-line compact form. Used by search, bookmarks and the side menu.
`src/components/SideMenu.tsx`	2.8 KB	The drawer's content: the app name and hadith count, the screen links, and the 3 newest bookmarks.
`src/theme/index.ts`	2.3 KB	Font names and files; `MAX_SCALE` (1.5 interface, 1.3 bars, 2 reader); line height 2; the **light and dark colour tokens** `background, surface, text, muted, border, prophet, speech`; `useColors()`.
`src/search/simplify.ts`	0.6 KB	Simplifies Arabic for search: writes out ﷺ, removes tashkeel and tatweel, and turns أ إ آ ٱ into ا, ى into ي and ة into ه. It must match the Python version in `book_data/tools/arabic.py`.
`src/search/simplify.test.ts`	0.3 KB	Runs the shared cases in `database/simplify.json`.
`src/text/count.ts`	0.9 KB	Arabic number agreement: «حديث واحد، حديثان، 3 أحاديث، 11 حديثًا، 100 حديث», and the same for باب.
`src/text/count.test.ts`	0.7 KB	Tests for the counting rules.

assets: images and fonts

File	Size	What it is for
`assets/icon.png`	503 KB	The 1024×1024 app icon: v1's gold calligraphy (`#FFD700`) on `#111318`.
`assets/android-icon-foreground.png`	62 KB	The Android adaptive-icon foreground: the gold calligraphy on a transparent background.
`assets/android-icon-monochrome.png`	50 KB	The single-colour layer for Android 13+ themed icons.
`assets/splash-icon.png`	69 KB	Splash art for light mode: the calligraphy in **green** `#0B6E5F`, on `#FBFCFC`.
`assets/splash-icon-dark.png`	70 KB	Splash art for dark mode: the calligraphy in **green** `#5CC9B4`, on `#0F1615`.
`assets/favicon.png`	1 KB	Expo's default blue web favicon, from the project template. Unused, because web isn't a platform.
`assets/fonts/ScheherazadeNew-Regular.ttf, -Bold.ttf`	332 + 593 KB	The hadith text font (SIL), chosen because it keeps every tashkeel mark clear.
`assets/fonts/OFL-ScheherazadeNew.txt`	4.5 KB	Its SIL Open Font License, with the reserved names "Scheherazade" and "SIL".
`assets/fonts/Cairo-Regular / SemiBold / Bold.ttf`	3 × 91 KB	The interface font, carried over from v1.

database, plugins, scripts

File	Size	What it is for
`database/riyad.db`	4.9 MB	The content database the app ships with: 20 books, 367 chapters, 1,898 hadith. The text is stored as speaker-tagged pieces, with a simplified search copy and an FTS5 trigram index that reads that copy instead of storing its own. The word-meaning, benefit and narrator tables are empty until content steps 5–6. Schema version 2.
`database/README.md`	8 KB	The database's tables, common queries, the exact search rules, and how to bundle it with Expo.
`database/simplify.json`	1.7 KB	21 shared test cases: both the Python builder and the app's `simplify()` must pass them.
`plugins/withLinksDuringStartup.js`	1.8 KB	An Expo config plugin that edits the generated Android `MainActivity.kt` (`onNewIntent` → `setIntent`), so a deep link that arrives while JavaScript is still starting opens its hadith instead of being dropped. It fails loudly if a newer template no longer matches.
`scripts/build-preview-db.js`	5.4 KB	`npm run preview-db`: copies `riyad.db` into `.preview/` and fills the meanings and benefits with v1's commentary (matched by v1 id), marks it as a preview, and writes a hash stamp. Test builds only: never committed, never released.
`scripts/v1-commentary.js`	3.5 KB	Parses v1's tagged commentary (`<p0>` headings, `<p1>` words, `<bullet>`) into word meanings and benefits.
`scripts/v1-commentary.test.js`	5.4 KB	Tests for the parser on v1's different formats.

.maestro: device tests

File	Size	What it is for
`.maestro/reader.yaml`	2.7 KB	Right-to-left swiping, the bounce at the first page, vertical drags not taken as taps, tap for full screen, links (including a number past the end, which opens 1898), and reopening at the last hadith.
`.maestro/search.yaml`	1.4 KB	Search by number, reopening search from the menu with the keyboard ready, and word search (iPhone only, because Maestro can't type Arabic on Android).
`.maestro/books.yaml`	1.3 KB	Opening a book and chapter lands on its first hadith (681). The current chapter is marked, and the list scrolls to a chapter far down (hadith 1700).
`.maestro/bookmarks.yaml`	1 KB	Bookmark from the reader, open it from the side menu, then remove it on the bookmarks screen.
`.maestro/settings.yaml`	1.6 KB	The font size and theme survive a restart, the smallest size disables "smaller", About opens, and share opens the share sheet.
`.maestro/links.yaml`	0.4 KB	A link that arrives during startup opens its hadith.
`.maestro/page-line.yaml`	1 KB	The printed-page line shows only when the commentary isn't v1's preview.

book_data: the content pipeline

File	Size	What it is for
`book_data/README.md`	20 KB	The pipeline's manual: the sources (v1's database, the scans of *Nuzhat al-Muttaqin*, OCR text, sunnah.com, hadith-api, Tesseract), the 7 content steps and their status, the JSON format, setup, every tool command, and the "long run".

File	Size	What it is for
`book_data/CLAUDE.md`	1.6 KB	Rules for Claude Code sessions in the pipeline: the printed book is the truth, never fix a misprint silently, compare Arabic only through the tools, and decisions go in git.
`book_data/docs/audit.md`	8.5 KB	Step 1: 30 v1 hadith compared with the book. The matn is good (7 have a mistake), the commentary is an edited version of the book's (26 differ, 6 change the meaning), and the numbering drifts.
`book_data/docs/numbering.md`	6 KB	Step 3: how every hadith got the book's number (1–1,898) and page, every chapter its number, and the titles their hadith.
`book_data/docs/matn.md`	20 KB	Step 4 (in progress): how each matn is compared with the OCR and the hadith collections, then read against a crop of the scan in Claude Code rounds, and the rules those rounds follow.
`book_data/content/numbering.json`	6 KB	Where v1 and the book don't line up one to one: hadith v1 lacks, merged entries, a repeated text, and the chapter v1 lacks.
`book_data/content/matn/NN.json (13 files)`	0.1–145 KB	Step 4's decisions, one file per book: fixes (with the reason: `variant`, misprint, ...) keyed by hadith number, so they can be applied again after a rebuild. Book 01 has 144 hadith with decisions. An entry with no changes means checked, nothing to fix.
`book_data/content/audit/NNNN.json (30 files)`	1–5 KB each	The audit: for each of 30 v1 hadith (0008, 0084, 0159 ... 1866) the pages, the book's printed text and how v1 differs. The test set for steps 4–5.

book_data/content/hadith: the source of truth (one file per book)

Each file holds the book's title and its chapters. Each chapter has its book number, title, v1 id and its hadith: `id, old_id, page, title, matn[] (role + text), meanings, benefits, corrections, notes`. Sizes range from 2.5 KB (book 10) to 1.07 MB (book 01).

File	Book	Chapters	Hadith	Numbers
`01.json`	الكتاب الأول	81	680	1–680
`02.json`	كتاب الأدب	16	47	681–727
`03.json`	كتاب أدب الطعام	17	51	728–778
`04.json`	كتاب اللباس	9	35	779–813
`05.json`	كتاب آداب النوم والاضطجاع والقعود والمجلس والجليس والرؤيا	4	31	814–844
`06.json`	كتاب السلام	13	49	845–893
`07.json`	كتاب عيادة المريض وتشييع الميت والصلاة عليه وحضور دفنه...	22	62	894–955
`08.json`	كتاب آداب السفر	14	35	956–990
`09.json`	كتاب الفضائل	51	278	991–1268
`10.json`	كتاب الإعتكاف	1	3	1269–1271
`11.json`	كتاب الحج	1	14	1272–1285
`12.json`	كتاب الجهاد	7	91	1286–1376
`13.json`	كتاب العلم	1	17	1377–1393
`14.json`	كتاب حمد الله تعالى وشكره	1	4	1394–1397
`15.json`	كتاب الصلاة على رسول الله ﷺ	1	11	1398–1408
`16.json`	كتاب الأذكار	6	57	1409–1465
`17.json`	كتاب الدعوات	4	46	1466–1511

File	Book	Chapters	Hadith	Numbers
18.json	كتاب الأمور المنهي عنها	115	298	1512–1809
19.json	كتاب المنثورات والملح	1	61	1810–1870
20.json	كتاب الاستغفار	2	28	1871–1898

book_data/tools: the Python tools

File	Size	What it is for
tools/build_db.py	8 KB	Builds database/riyad.db from content/: tables, speaker pieces, search text, the FTS5 index and the info table. The same content always gives the same file.
tools/arabic.py	2 KB	Shared Arabic normalisation (normalize_word, simplify), the Python twin of src/search/simplify.ts.
tools/sources.py	6 KB	Downloads and opens the inputs: the scans PDF, the OCR text, and the v1 database.
tools/convert_v1.py	18 KB	Steps 2–3: turns v1's database into content/hadith/ (speaker pieces from v1's tags, ﷺ glyph, ordinals removed), with the book's numbering.
tools/numbering.py	14 KB	Step 3: aligns v1 with the OCR text to find the book's number for each hadith, and checks the mapping.
tools/chapters.py	6 KB	Step 3: matches v1's chapters to the book's numbered chapter headings.
tools/locate.py	4 KB	Finds the PDF pages where each v1 hadith and its commentary are printed.
tools/audit.py	8 KB	Step 1: picks the 30-hadith sample, shows v1 next to the OCR, and writes and sums up the audit records.
tools/matn.py	56 KB	Step 4, the largest tool: compares each matn with the OCR, sunnah.com and the collections, lists the words in doubt, prepares each round's hadith with scan crops, and applies the decisions.
tools/ocr.py	5 KB	Runs Tesseract over the hadith pages, keeping each word's position, so a matn can be cropped from the scan.
tools/fetch_ocr.sh	0.6 KB	Unpacks Tesseract from Debian packages into .cache/ without installing anything system-wide.
tools/render_pages.py	2 KB	Renders PDF pages to PNG.
tools/zoom.py	6 KB	Renders zoomed crops of a page, to read small marks.
tools/long_run.sh	6 KB	Keeps step 4 going: starts one Claude Code session per round of up to 12 hadith, and waits out usage limits.
tools/long_run.py	8 KB	Helpers for the long run: reads the usage limits and sums up each round for the log.
tools/requirements.txt	16 B	Python dependency: pymupdf==1.28.2.

v1: the old Kotlin app (reference only)

File	Size	What it is for
v1/app/src/main/java/…/MainActivity.kt	6 KB	v1's single activity: sets up Compose, the theme and navigation.
…/data/AppPreferences.kt	3.5 KB	v1 settings (font size, dark-mode toggle, reading progress, bookmarks) in Jetpack DataStore.

File	Size	What it is for
…/data/DatabaseHelper.kt, …/repository/RiyadSalheenRepository.kt	2 + 6 KB	Opens v1's bundled database and queries books, chapters, hadith and search.
…/model/Book.kt, Door.kt, Hadith.kt	< 1 KB	v1 data classes (a "Door" is a chapter, باب).
…/ui/screens/HadithDetailScreen, NavigationScreen, SearchScreen, BookmarksScreen, AboutScreen (.kt)	9–18 KB each	v1's screens. They are the reference for how v2's features behave.
…/ui/components/NavigationDrawer, HtmlText, FontSizeDialog, HideSystemBars, LoadingContent (.kt)	0.7–12 KB	v1's shared composables: the drawer, the tagged-text renderer, the font dialog and full screen.
…/ui/theme/Color.kt, Theme.kt, Type.kt	3–5 KB	v1's Material 3 theme: the default **blue** palette (`#415F91`) and font setup.
…/ui/viewmodel/MainViewModel.kt	6 KB	v1's app state.
v1/app/src/androidTest/…/RepositoryTest.kt, src/test/…/ExampleUnitTest.kt	3 + 0.3 KB	v1's instrumented repository test and the template unit test.
v1/app/src/main/AndroidManifest.xml	1 KB	v1's manifest.
v1/app/src/main/assets/databases/riyad_salheen.db	15.3 MB	v1's content database (HTML-colour tags turned into `<p0>/<p1>/<bullet>`). An input to the pipeline and the preview database.
v1/app/src/main/res/drawable/*.xml (24 files)	small	v1's vector icons (arrows, bookmark, search, share, settings, …) and the launcher foreground, the vector source that v2's icon was rebuilt from.
v1/app/src/main/res/font/*.ttf (9 files)	—	Amiri (regular and bold), Cairo (3 weights), Noto Sans Arabic (4 weights).
v1/app/src/main/res/mipmap-*/ic_launcher*.webp/.xml (12 files)	small	v1's launcher icons for each screen density, and the adaptive-icon definitions.
v1/app/src/main/res/values/*.xml, xml/*.xml (6 files)	small	Colours, strings, themes, the launcher background, and backup rules.
v1/app/src/main/ic_launcher-playstore.png	47 KB	v1's Play Store icon.
v1/app/build.gradle.kts, proguard-rules.pro, .gitignore	3 KB	v1's module build (minify and shrink on, `versionCode 1`), R8 rules.
v1/build.gradle.kts, settings.gradle.kts, gradle.properties, gradle/libs.versions.toml, gradle/wrapper/*, gradlew, gradlew.bat, .gitignore	—	v1's Gradle project and wrapper.
v1/riyad_keystore.jks	2.7 KB	The Play Store signing key, committed on purpose (owner's decision); its passwords are kept out of git. v2 must be signed with it to update v1.

File	Size	What it is for
`v1/docs/screenshots/*.png (6)`	0.1–0.3 MB	v1 screenshots: Home, Navigation, NavigationDrawer, Search, Favourite, About.
`v1/dp_update/main.py`	17 KB	The Python script that cleaned the original HTML-formatted database into v1's tagged text.
`v1/dp_update/riyad_salheen.db, hadith_processing_log.txt`	15.3 MB + 8 KB	The original HTML-formatted source database and the script's log.

4 How the app works

4.1 What the user sees

The app is **Arabic only and right-to-left**. It opens on the **reader**, one hadith per page, at the last hadith read. A side menu (from the right) leads to six screens:

Screen	What it does
القراءة Reader	Swipe to turn pages like an Arabic book: drag right for the next hadith. Tap to hide the bars for full screen. The bar has the menu, the book title, search, share and bookmark. Above each hadith: the chapter, «الحديث رقم N» and the short title. Below it: word meanings and benefits (when the database has them) and the printed page.
الكتب والأبواب Books & chapters	20 books that expand into chapters. The current book is open and its chapter marked; a chapter opens its first hadith.
البحث Search	Type a word (tashkeel and alif/ya/ta-marbuta forms are ignored, and part of a word matches) or a hadith number in any digits. Up to 100 results, in book order.
المفضلة Bookmarks	Newest first. Remove one or clear all, each with a confirmation. The 3 newest also show in the side menu.
الإعدادات Settings	Font size 14–30 with a sample; theme System / Light / Dark.
حول التطبيق About	The book, the author, the features, the version, and contact by email or GitHub.

Planned but not built yet: “tap an explained word” to see its meaning, and “tap a narrator’s name” to read the biography. Both wait for content steps 5–6, which will add the commentary, the narrators and the links between words and text. Until then the release database has no word meanings or benefits, so a release built today would show only the hadith text and the page line.

4.2 What happens at startup

1. The native splash screen stays up (`SplashScreen.preventAutoHideAsync`).
2. The navigator mounts straight away, so an early deep link (`riyadalsalheen:///?id=12`) has somewhere to go. On Android, `withLinksDuringStartup` keeps that link for Expo Router.
3. The five fonts load.
4. `ContentProvider` copies the bundled `riyad.db` to `riyad—<hash>.db` (only the first time, or after an update brings a new database), checks `PRAGMA user_version` = 2, deletes the previous copy, and reads the `info` table.
5. The theme setting from `user.db` is applied to the phone’s own views, and the splash screen hides.
6. The reader opens at the hadith from the link, or the last one read, or hadith 1.

4.3 The reader in detail

- **Paging:** `Pager` keeps one horizontal strip and animates only its position, so a page turn re-renders nothing. It renders 5 pages (current ± 2). The README reports 0.3% janky frames over 30 fast swipes on a Galaxy A17.
- **Text styling:** each hadith is stored as pieces with a speaker `role`: `narration` (plain), `prophet` (colour + bold), `speech` and `gloss` (a quieter second colour). The roles come from v1 and haven’t been checked yet; a later content pass fixes them.
- **Full screen:** a tap (one that doesn’t move more than 10 px and lasts under 300 ms) hides the header, the status bar and the navigation bar. The text doesn’t move, because the pages keep the largest safe-area margins seen.
- **Accessibility:** only the visible page is exposed to screen readers. “Next/previous hadith” actions turn pages, and the new number is announced. Text sizes respect the phone setting up to a cap (reader 2×, interface 1.5×, bars 1.3×).

- **Position:** every page turn saves the hadith number in `user.db`; reopening the app returns there.

4.4 Data and storage

Store	Contents	How it changes
`riyad.db` (bundled, read-only)	Books, chapters, hadith, text pieces, search text + FTS5 trigram index; empty meanings, benefits and narrators tables.	Rebuilt by `build_db.py` and committed. A new app version with a new file is copied fresh, because the copy is named after the file's hash.
`user.db` (on the phone)	`settings` (font size, theme, last hadith, current content file) and `bookmarks` (hadith number + time).	Only through append-only migrations. Content updates never touch it. v1's saved data is not carried over (owner's decision).
In memory	Up to 30 recently read hadith; the bookmark and settings stores.	Bookmark and settings changes notify every screen at once.

Search works like this. A query of digits (Western, ٩–٠ or ٠–٩) opens that hadith number. Any other query is simplified exactly as the build script simplified the text, needs at least 3 letters (the trigram minimum), and is matched anywhere in the text with `MATCH '"…"'`. The app fetches 101 rows to know whether there are more than 100.

4.5 Build variants

Variant	How it is built	Id / name	Database
Development	`expo run:android|ios` or `eas build -p … --profile development`; includes the dev client	`com.nad.riyadalsalheen.dev`, «… (تجربة)»	Preview DB with v1 commentary if it's current, otherwise the real one
Preview	`eas build --profile preview`; Android: a release **APK**	same `.dev` id	same as above
Production	`eas build --profile production` (`APP_VARIANT=production`); Android: an **app bundle (.aab)**	`com.nad.riyadalsalheen`	Always the real `riyad.db`

4.6 The content pipeline

The printed book (*Nuzhat al-Muttaqin*, 14th edition) is the reference. Steps 1–3 (audit, convert v1, number as printed) are done. Step 4 (check every matn against the scans) is under way: 164 of 1,898 hadith are checked in the committed database, and more in `content/matn/`. Steps 5–7 (transcribe the commentary, transcribe and link the narrators, owner review) are still to come. The README's plan is: content first, then app, then release.

4.7 Tests and checks

Check	Result on bd1546f (run for this report)
`npm test` (Jest, 8 suites)	**65 / 65 passed**, including SQL run against the real database.
`npm run typecheck` (2 configs)	**No errors.**
`npm run lint`	**No problems.**
Maestro, 7 flows	Not run here (needs a device); the README reports them passing on the iPhone simulator and a Galaxy A17 (2026-11-04).

5 Review: why the Android APK is about 100 MB, and how to shrink it

You're right that 100 MB is far too much for an app that is mostly text. But very little of that size comes from this app. I bundled the production JavaScript and assets (`npx expo export --platform android`), generated the Android project (`npx expo prebuild`), and read the Gradle settings it produces. The JavaScript, the database and the fonts together are about 11 MB uncompressed and about 8 MB inside the APK. Almost all the rest is native code: compiled libraries (`.so` files) for React Native, the Hermes engine, Reanimated and Worklets, Gesture Handler, Screens, Expo's modules and SQLite.

5.1 What is inside a 100 MB universal APK

Part	Size in the APK	How we know
Native libraries for **4 CPU types** (`armeabi-v7a, arm64-v8a, x86, x86_64`), stored **uncompressed**	≈ 80–90 MB (≈ 20 MB per CPU type)	Defaults read from the generated project: `reactNativeArchitectures=armeabi-v7a,arm64-v8a,x86,x86_64` and `expo.useLegacyPackaging=false` (native libraries not compressed). The per-CPU size is an estimate for this set of libraries.
JavaScript bundle (Hermes bytecode), stored uncompressed	4.2 MB	Measured. `enableBundleCompression` is `false` by default (it is faster to start).
`riyad.db`	≈ 1.9 MB (4.9 MB raw)	Measured: it compresses to 1.87 MB. The file is already lean: the search index reads the hadith table's copy instead of storing a second one.
Fonts: Scheherazade ×2, Cairo ×3, and expo-symbols' Material Symbols icon font	≈ 0.9 MB (2.2 MB raw)	Measured. The icon font alone is 0.97 MB raw.
Java/Kotlin code (`classes.dex`) and Android resources	a few MB	Estimate. R8 minification is already on in SDK 58's template (`android.enableMinifyInReleaseBuilds=true`); resource shrinking is off.

> **A phone only ever uses one CPU type.** Nearly every Android phone in use today is `arm64-v8a` (your Galaxy A17 is). `x86` and `x86_64` exist almost only for emulators and a few Chromebooks. So a universal APK carries about three copies of the native code that the phone never loads.

Two other things can make an APK bigger still. A **development build** (`npx expo run:android`, or the `development` profile) is a debug build: it includes the dev client, no optimisation, and debug native libraries, so it is much bigger than a release build. Don't judge the app's size from it. The `preview` profile builds a release APK, but a universal one.

5.2 What to change, in order of impact

#	Change	Effect	Cost
1	**Release on Google Play as an app bundle** (`.aab`). The `production` profile already does this. Play then builds a split APK for each phone: one CPU type, one screen density.	What users download is roughly **15–25 MB** instead of 100 MB (estimate). Nothing to change; just measure the right file.	None

#	Change	Effect	Cost
2	**Build test APKs for `arm64-v8a` only** with `expo-build-properties`' `buildArchs` (snippet below).	Preview and development APKs drop to about **30 MB** (estimate), and they build faster.	Won't install on an x86 emulator; use an arm64 emulator (Apple Silicon) or a phone.
3	**Turn on resource shrinking**: `enableShrinkResourcesInReleaseBuilds: true` (minify is already on).	Removes unused Android resources: a small saving.	Test a release build once.
4	*Optional, for sideloaded APKs:* `useLegacyPackaging: true` stores native libraries compressed.	The APK file gets much smaller (native code compresses by roughly half).	Installs a bit slower and takes more space once installed; makes no difference to Play downloads.
5	*Optional:* `enableBundleCompression: true` compresses the 4.2 MB JS bundle to about 1.7 MB.	−2.5 MB	Slightly slower app start. Not recommended for the Play build.
—	Not worth it: subsetting fonts, shrinking the database, or replacing icons.	Under 1 MB in total.	—

Install the plugin with `npx expo install expo-build-properties` (version 58.0.12 matches SDK 58; I checked the option names in that package's own type definitions). Then, in `app.config.js`:

```
const sizeOptions = (android) => ['expo-build-properties', { android: {
enableShrinkResourcesInReleaseBuilds: true, ...android } }];

module.exports = ({ config }) => {
  if (process.env.APP_VARIANT === 'production') {
    // The Play build: an app bundle, so Play splits it per CPU type itself.
    return { ...config, plugins: [...config.plugins, sizeOptions({})] };
  }
  return {
    ...config,
    name: `${config.name} (تجربة)`,
    scheme: `${config.scheme}-dev`,
    ios: { ...config.ios, bundleIdentifier: `${config.ios.bundleIdentifier}.dev` },
    android: { ...config.android, package: `${config.android.package}.dev` },
    // Test builds: only the CPU type of real phones (and Apple-Silicon emulators).
    plugins: [...config.plugins, sizeOptions({ buildArchs: ['arm64-v8a'] })],
  };
};
```

How to check your own APK: an APK is a zip file. `unzip -l app.apk | sort -n | tail -25` lists the largest entries; you should see `lib/arm64-v8a/…`, `lib/x86_64/…` and so on, near the top. Android Studio's *Build → Analyze APK* shows the same as a tree. For the app bundle, `bundletool get-size total --apks=…` gives the real download size per device.

Related, but about build time rather than app size: EAS uploads the whole project for every build, and `v1/` alone holds about 32 MB (two copies of v1's 15 MB database and the screenshots). An `.easignore` file (same syntax as `.gitignore`; it replaces `.gitignore` for uploads, so copy its rules in) that also lists `v1/` and `book_data/` makes each upload much smaller.

6 Review: design and colours

6.1 The brand is gold on black, and the app doesn't use it



Left: the app icon (`assets/icon.png`), gold `#FFD700` on `#111318`. Middle and right: the splash screens (`splash-icon.png`, `splash-icon-dark.png`), which redraw the same calligraphy in green.

You noticed it correctly: the highlighted text is green. The token `prophet` is teal green in light mode and mint in dark mode. Here is the whole current palette:

Token	Light	Dark	Used for
background	`#FBFCFC`	`#0F1615`	Screens, header, drawer
surface	`#F1F5F4`	`#172120`	Pressed rows, search field
text	`#14211F`	`#E4ECEA`	Body text
muted	`#55625F`	`#9AABA7`	Chapter, number, notes
border	`#D6DFDC`	`#2A3735`	Hairlines
prophet	`#0B6E5F`	`#5CC9B4`	The Prophet's words **and** every accent (22 uses)
speech	`#3D4F6B`	`#A9B9D6`	Other people's words

Three separate problems come from this:

1. **No link to the logo.** The user taps a gold-on-black icon and gets a green splash screen, then a cool grey-green app. The dark background (`#0F1615`) is a different black from the icon's (`#111318`). v1 had the same disconnect: it used Material 3's default blue (`#415F91`).
2. **One colour means everything.** `colors.prophet` is used 22 times in 10 files: the Prophet's words, but also «لغة الحديث» and «أفاد الحديث», settings and About headings, the About title, the active menu item, numbers in lists, book icons, the current chapter, the bookmark icon, «مسح الكل», link icons and loading spinners. The colour should tell the reader "these are the Prophet's words ﷺ", but in the reader the section headings below the text are the same green.
3. **Cool colours around warm content.** Calligraphy, gold and classical Arabic type all suggest a warm, paper-and-ink feel; the blue-green greys work against that.

6.2 Why you can't simply use the logo's gold

Pure gold `#FFD700` works beautifully on the dark background (13.3 : 1 contrast). On a white or paper background it is **1.36 : 1**, which is unreadable for text and fails even the 3 : 1 minimum for icons. So the light theme needs a **deeper bronze-gold** for text and icons, and the bright gold is kept for dark mode and the logo itself. This is the usual way to adapt a gold brand to a light theme.

6.3 Proposed palette

Both proposals share the same base: warm paper and ink in light mode, the icon's own `#111318` in dark mode, and **gold as the accent**. They also split the overloaded token in two: `accent` for the app's chrome (buttons, icons, active items, links, spinners), and `prophet` only for the Prophet's words. Section headings become text-coloured, with a short gold bar, so in the reader colour marks only the speakers.

- **Proposal A, "Gold & ink" (recommended):** the Prophet's words are in gold, bold. It ties the most important text to the logo, which is itself gold, and in dark mode it looks exactly like the icon.
- **Proposal B, "Manuscript":** the Prophet's words are in a deep rubric red, as many printed hadith books and manuscripts mark them, with gold kept for the chrome. It is the more classical look.

Token	Light	AA contrast	Dark	AA contrast
background	`#FAF7F0` paper	—	`#111318` = icon	—
surface	`#F2EDE1`	—	`#1A1D24`	—
text	`#1C1A16` ink	16.2 : 1	`#ECE7DA`	15.1 : 1
muted	`#6A6255`	5.6 : 1	`#A39C8E`	6.8 : 1
border	`#E2DACA`	—	`#2A2F3A`	—
accent (new)	`#8A6500` gold	5.0 : 1	`#E9C54A` gold	11.1 : 1
prophet A	`#7A5600` bold	6.2 : 1	`#F2CF5B` bold	12.3 : 1
prophet B	`#9A2B2B` bold	7.1 : 1	`#F08C80` bold	7.8 : 1
speech	`#3F4C66`	8.1 : 1	`#AEBBD3`	9.6 : 1

Contrast is against the background, computed with the WCAG 2 formula. AA needs 4.5 : 1 for normal text. In every proposal the Prophet's words are bold as well as coloured, so they still stand out for colour-blind readers and in greyscale.

6.4 Mockups: the reader with each palette

I recreated the reader screen in HTML from `HadithPage.tsx`, `HadithText.tsx` and `ReaderHeader.tsx` (same sizes, the app's own fonts) and showed hadith 181 from the test-build database. These are not device screenshots. Top row light, bottom row dark. Left to right: current, Proposal A, Proposal B.



Current · light



A "Gold & ink" · light



B "Manuscript" · light



Current · dark



A "Gold & ink" · dark



B "Manuscript" · dark

6.5 Making the change in code

The theme is already centralised, so the change is small. Edit the two objects in `src/theme/index.ts` and add an `accent` token:

```
const light = {
  background: '#FAF7F0', surface: '#F2EDE1', text: '#1C1A16', muted: '#6A6255', border: '#E2DACA',
  accent: '#8A6500',   // the brand's gold: buttons, icons, active items, links, spinners
  prophet: '#7A5600',  // only the Prophet's words, always bold (Proposal B: '#9A2B2B')
  speech: '#3F4C66',
};
const dark: Colors = {
  background: '#111318', surface: '#1A1D24', text: '#ECE7DA', muted: '#A39C8E', border: '#2A2F3A',
  accent: '#E9C54A', prophet: '#F2CF5B', speech: '#AEBBD3',   // Proposal B prophet: '#F08C80'
};
```

Then replace `colors.prophet` with `colors.accent` everywhere except `src/reader/HadithText.tsx` (21 of the 22 uses), and give the section titles in `HadithPage.tsx`, `settings.tsx` and `about.tsx` `colors.text` with a small accent bar. To match, update `app.json`:

- **Splash screen:** use the icon's art (gold on `#111318`) for both modes, so launching feels like the icon opening; or, for light mode, deep gold `#8A6500` art on `#FAF7F0`. Replace the two green `splash-icon*.png` files.
- **Splash** `backgroundColor` `#FBFCFC` / `#0F1615` → the new backgrounds.

6.6 Typography and layout

- **The fonts are a good choice.** Scheherazade New is made for fully vocalised text, and Cairo is a clean interface face. Keep both.
- **The hadith title uses the wrong font.** In the reader the short title (e.g. «نَعَمْ وَلَكِ أَجْرٌ») is drawn in Cairo Bold, but it is a phrase of the hadith with full tashkeel, the case that made you drop Amiri. Lists already draw the same titles in Scheherazade (`HadithRow.tsx`). Use `fonts.textBold` in `HadithPage.tsx` too, for consistency and clearer marks.
- **The chapter (باب) is the least visible line** (14 px, muted), but it is the context readers care about most after the hadith. Give it more weight, and make it a link to Books & chapters (see UX).
- **Justified text:** on phones, React Native justifies Arabic by widening the spaces (no kashida stretching). At large font sizes on narrow screens that leaves wide gaps between words. Compare with start-aligned text at sizes 26–30, or justify only below a certain size.
- **Brand moments:** put the gold calligraphy at the top of the side menu (instead of plain text «رياض الصالحين») and on the About screen. It is the app's most distinctive asset, and today it appears only in the launcher icon.
- **iPad:** `supportsTablet: true`, but the reader stretches the text across the whole width, so lines on an iPad are very long. Limit the reading column to about 680 pt and centre it.

7 Review: user experience

The basics are solid. The page turn feels like an Arabic book (swipe right for the next hadith), the app reopens where you left off, and search ignores tashkeel and accepts any digits. Bookmarks stay in sync everywhere, and the empty and loading states are handled. Accessibility is unusually well done for an app this size. These are the gaps, highest impact first:

Priority	Issue	Suggestion
High	**Search results don't show why they matched.** Each result shows the hadith's short title and its book and chapter, which often don't contain the searched word, so the user can't tell which result is the one they want.	Show a one-line snippet around the match with the matched word highlighted. (`database/README.md` already explains how: simplify the text word by word to find the match position.) In the reader, highlight the word on the opened page too.
High	**Tapping anywhere toggles full screen.** The planned "tap a word" and "tap a narrator" features need taps on the text. Accidental toggles while reading are also common.	Decide now: taps on linked words open their meaning or biography, and taps elsewhere toggle full screen; or move full screen to a button or a double tap. Changing `Pager`'s tap handling later is harder.
High	**The hadith text can't be selected or copied.** Share sends the whole hadith, but people often quote a sentence.	Add `selectable` to the text (test it with the pager's gestures), or a "copy" action next to share.
Medium	**No sense of place, and no quick jump.** The page says «الحديث رقم 181» but not out of how many, and jumping to a number means opening search.	Show «181 / 1898», or a thin progress line under the header. Make the number tappable to type a hadith number.
Medium	**The chapter line isn't interactive.**	Tap the chapter to open Books & chapters at it (that screen already scrolls to the current chapter). Consider "next/previous chapter".
Medium	**Removing a bookmark asks for confirmation every time** (as v1 did). It's a small, reversible action, and the extra dialog slows it down.	Remove at once and show a toast/snackbar with «تراجع» (Undo). Keep the confirmation only for «مسح الكل».
Medium	**Errors are shown raw, in English.** For example, the reader shows `riyad.db has schema 3, but the app is written for 2`, or an SQLite message, to Arabic readers.	Show an Arabic message with a «إعادة المحاولة» button, and keep the technical detail for logs or a small "details" line.
Medium	**Search stops at the first 100 results** with the note «أول 100 حديث», and there is no way to see more.	Load more as the list scrolls, or let the user narrow the search to a book.
Low	**First launch gives no hints.** New users don't know to swipe right for the next hadith, or that a tap hides the bars.	A one-time, dismissible hint on the first page.
Low	**The screen dims and locks during long reading.**	Keep the screen awake while the reader is open (`expo-keep-awake`), perhaps as a setting.
Low	**Bookmarking gives no feedback** beyond the icon changing.	A short toast «أضيف إلى المفضلة» or a light haptic.
Low	**Few reading settings.**	Consider line spacing, and possibly showing or hiding tashkeel (the content team should decide that one, since the tashkeel is the main content work).

What's already good, and worth keeping: the page turn matches how Arabic books are read; the share text now includes the hadith number; the System / Light / Dark setting fixes v1's inverted toggle; the Prophet's words are bold as well as coloured; text sizes are capped so big accessibility fonts don't break the layout; screen readers get page-turn actions; and test builds clearly label the unreviewed v1 commentary.

8 Review: code, build and release

8.1 Strengths

- **Clear structure and unusually good comments.** Almost every non-obvious line explains *why*, often citing the device bug it works around.
- **The content and user data are cleanly separated:** a read-only, hash-named content database, plus `user.db` with transactional, append-only migrations. Content updates can never wipe bookmarks.
- **Safety checks that matter:** the schema version check at startup, the preview database can never reach a release (enforced in `metro.config.js` and tested), and the search simplification is kept in step with Python by shared test cases.
- **Performance thinking:** the pager animates one value and renders 5 pages, there is a 30-hadith cache with shared in-flight loads, and the settings stores re-render only what changed.
- **Tests at three levels:** unit tests run the real SQL against the real database, there are config tests, and Maestro device flows cover the real gestures.

8.2 Issues and suggestions

Priority	Finding	Suggestion
High	**Unneeded Android permissions.** The Android manifest generated from this config asks for `INTERNET`, `READ/WRITE_EXTERNAL_STORAGE`, `SYSTEM_ALERT_WINDOW` ("display over other apps") and `VIBRATE`. The Play listing shows these, and they contradict "offline, no tracking".	In the production branch of `app.config.js`, set `android.blockedPermissions` to those four. Then test a release build without `INTERNET` as well: the app doesn't need it, and "no internet access" is a strong privacy signal. Keep them in dev builds, which need the network to reach Metro.
High	**`versionCode` is set by hand** (`appVersionSource: "local"`, `versionCode: 2`). Every upload to Play, including each internal test in P6, needs a higher number.	Use `"appVersionSource": "remote"` in `eas.json` and `"autoIncrement": true` on the production profile.
Medium	**The About links point to v1's accounts:** `github.com/1992please/riyad-alsalheen` and `1992please@gmail.com`. The project now lives at `github.com/nadernour/riyad`.	Confirm which contact and repository you want public, and update `about.tsx`.
Medium	**Unchecked speaker roles show in the colours.** For example, in hadith 181 the opening « of the Prophet's words belongs to the narration piece before them, so the quote mark has the wrong colour.	Already planned (the roles pass after step 4). Add a check to `build_db.py` that flags a « or » split away from the `prophet` piece it belongs to.
Medium	**iPad is enabled but not designed for**, and the app is portrait-only.	Either design for iPad (a centred reading column, and probably landscape) or set `supportsTablet: false` for the first release. Check Apple's orientation and multitasking requirements before the TestFlight upload.
Low	**The repository grows with every database build:** each commit of `riyad.db` adds 4.9 MB, and `v1/` holds two copies of a 15 MB database.	Git LFS for `*.db` keeps them committed (the owner's decision) without bloating every clone. Add an `.easignore` for build uploads (see 5.2).
Low	`assets/favicon.png` is Expo's blue template icon, unused because web isn't a platform.	Delete it, or replace it with the gold mark.
Low	**The dev client is in `dependencies`.** It is meant to stay out of release builds.	When you have a release APK, open it in Android Studio's APK Analyzer and check that no `expo.modules.devlauncher` classes are inside.

Priority	Finding	Suggestion
Low	The side menu loads summaries of *all* bookmarks to show 3.	Fine at today's sizes. If people collect hundreds of bookmarks, query only the first few that exist.

9 Suggested order of work

Now, before the P6 test builds

1. Size: add `expo-build-properties` (arm64-only test builds, resource shrinking) and an `.easignore`. Measure the preview APK and the production AAB (`bundletool get-size`).
2. Release config: blocked permissions, remote and auto-incremented `versionCode`, About contact links, an iPad decision.
3. Colours: pick Proposal A or B, add the `accent` token, re-point the 21 chrome uses, and redo the splash screen and side-menu header.

Next

4. Search snippets with highlighting, and the Arabic error screen.
5. Decide how taps work (full screen vs tap-a-word/narrator) before content steps 5–6 land.
6. Selectable text or a copy action, the position indicator and jump-to-number, the tappable chapter line, and undo for bookmark removal.

Later

7. First-run hint, keep-awake, reading settings, the iPad layout, and Git LFS for databases.

10 Appendix: how the numbers were measured

- **Checkout:** `origin/v2-app` at `bd1546f`, in a separate worktree; `npm ci` with Node 22.23.
- **Checks:** `npx jest` (8 suites, 65 tests passed), `npm run typecheck` and `npm run lint` (both clean).
- **JavaScript and assets:** `APP_VARIANT=production npx expo export --platform android` produced a 4.17 MB Hermes bundle and 37 assets: `riyad.db` 4.93 MB, the five fonts 1.20 MB, and the Material Symbols font 0.97 MB. The compressed sizes are zlib level 9 (what a zip/APK entry roughly gets).
- **Android defaults:** `APP_VARIANT=production npx expo prebuild --platform android --no-install` wrote `gradle.properties` with `reactNativeArchitectures=armeabi-v7a,arm64-v8a,x86,x86_64`, `android.enableMinifyInReleaseBuilds=true` and `expo.useLegacyPackaging=false`; `app/build.gradle` defaults `enableBundleCompression` and `shrinkResources` to `false`. The generated manifest lists the permissions in 8.2.
- **Not measured:** a real APK could not be built here (no Android SDK, and Google's download servers are not reachable from this environment). So the native-library sizes and the after-fix totals are estimates. The `unzip -l` check in 5.2 confirms them on your own APK in a minute.
- **Database:** per-table sizes from SQLite's `dbstat`: text pieces 1.94 MB, FTS index 1.50 MB, the hadith table (with search text) 1.22 MB.
- **Colours:** read from `src/theme/index.ts`, `app.json` and the PNGs (the icon foreground's solid pixels are exactly `#FFD700`). Contrast ratios use the WCAG 2.x relative-luminance formula.